



PORTAFOLIO ACADÉMICO EN GITHUB

Guía de implementación

Control de Versiones			
Versión	Fecha	Responsable(s)	Descripción
1.0	27/04/2026	Vanessa Jurado Allan Avendaño	Versión Inicial del documento

Contenido

1. Objetivo de la Actividad	3
2. Estructura y Nomenclatura	3
3. Paso a Paso: Creación del Repositorio Principal	3
4. Flujo de Trabajo por Proyecto	4
Principios fundamentales	5
Qué comentar (y con qué nivel de detalle)	5
Qué evitar	5
Estilo, formato y convenciones	6
Mantenimiento y calidad	6
💡 Regla de oro práctica	6
5. Plantilla Obligatoria de README.md (Por Proyecto)	7
6. Estadísticas y Métricas de Progreso	8
7. Comandos Clave de Git (Referencia Rápida)	9
8. Entrega, Revisión y Rúbrica	9
9. Checklist Final de Verificación	10

1. Objetivo de la Actividad

El propósito de esta actividad es que construyas un **portafolio profesional documentado** que compile todos tus proyectos académicos a lo largo de tu carrera. Con ello lograrás:

- Aplicar control de versiones (Git/GitHub) con estándares de la industria.
- Documentar proyectos con estructura técnica clara y reproducible.
- Rastrear tu progreso mediante métricas reales de desarrollo.
- Presentar tu trabajo organizado por periodo y materia, creando un recurso vivo que podrás usar en procesos de selección, pasantías o entrevistas técnicas.

2. Estructura y Nomenclatura

Tu portafolio se organizará en carpetas por materia usando la nomenclatura exacta:

Periodo - Nombre de la Materia

Ejemplo: '2026-1 - Lenguajes de Programación'

Cada materia contendrá uno o más proyectos. La estructura general será:

tu-usuario.github.io/

└─ README.md ← Página principal del portafolio

└─ 2026-1 - Lenguajes de Programación/

| └─ proyecto-01-analizador-lexico/ ← Enlace o índice al repo externo

| └─ proyecto-02-interpreter/

└─ 2026-2 - Bases de Datos/

└─ ...

Nota importante: Cada proyecto debe vivir en **su propio repositorio independiente**. El portafolio central (`tu-usuario.github.io`) solo actúa como índice y vista previa.

3. Paso a Paso: Creación del Repositorio Principal

1. Crea el repositorio Pages

- Inicia sesión en GitHub → `New repository`

- Nombre exacto: `tu-usuario.github.io` (reemplaza con tu usuario)
- ☒ `Public`
- ☒ `Add a README file`
- Click en `Create repository`

2. Activa GitHub Pages

- Ve a `Settings` → `Pages`
- En `Source`, selecciona `Deploy from a branch`
- Rama: `main` | Carpeta: `/ (root)`
- Click en `Save` → Espera 1-2 min. Tu portafolio estará disponible en `https://tu-usuario.github.io`

3. Clona y prepara la estructura

```
git clone https://github.com/tu-usuario/tu-usuario.github.io.git
cd tu-usuario.github.io
mkdir "2026-1 - Lenguajes de Programación"
```

4. Crea el índice de la materia

Dentro de la carpeta de la materia, crea un archivo `README.md` que liste tus proyectos con enlaces directos a sus repositorios oficiales. GitHub Pages renderizará automáticamente estos archivos en formato web.

Su cuenta de github debe ser creada con correo institucional.

4. Flujo de Trabajo por Proyecto

Para cada proyecto nuevo:

1. **Crea el repositorio del proyecto** en GitHub con nombre descriptivo (ej: `parser-regex-python`).

2. **Clónalo y desarrolla:**

```
git clone https://github.com/tu-usuario/nombre-proyecto.git
cd nombre-proyecto
git checkout -b feature/desarrollo-inicial
```

Durante el desarrollo usa comentarios. Puedes utilizar las siguientes prácticas:

Principios fundamentales

1. **Comenta el *por qué*, no el *qué*:** El código ya expresa qué hace. Los comentarios deben justificar decisiones de diseño, restricciones de negocio, workarounds o elecciones técnicas no obvias.
2. **El código autodocumentado primero:** Antes de comentar, refactoriza para que nombres de variables, funciones y estructuras expresen su intención. Usa comentarios solo cuando la claridad léxica no sea suficiente.
3. **Un comentario desactualizado es peor que ninguno:** Si el código cambia, el comentario debe actualizarse o eliminarse. Comentarios obsoletos generan desconfianza y bugs por malas interpretaciones.

Qué comentar (y con qué nivel de detalle)

4. **Documenta interfaces públicas y APIs:** Especifica parámetros, tipos, valores de retorno, excepciones lanzadas, precondiciones y postcondiciones. Usa el formato estándar del lenguaje (JSDoc, Python docstrings, JavaDoc, XML docs, etc.).
5. **Explica complejidad algorítmica o lógica no intuitiva:** Optimizaciones, atajos matemáticos, patrones inusuales o soluciones a edge cases deben llevar un comentario breve que describa la intención.
6. **Registra suposiciones y limitaciones explícitas:** Indica dependencias implícitas, entornos requeridos, casos no cubiertos, trade-offs aceptados o compatibilidad con versiones externas.
7. **Añade referencias trazables cuando corresponda:** Enlaza a tickets, RFCs, debates de equipo, especificaciones o reportes de bug que justifiquen la implementación. Evita enlaces muertos; usa IDs internos si es necesario.

Qué evitar

8. **Elimina comentarios redundantes o obvios:** `// cierra el archivo` después de `file.close()` añade ruido. Si el código es claro, no comentes.
9. **No uses comentarios para almacenar código muerto:** El historial de versiones (Git, etc.) ya guarda el contexto. Comentar bloques completos sin justificación temporal o de depuración genera deuda visual.
10. **Evita el humor, sarcasmo o anécdotas personales:** Los comentarios son documentación técnica. Deben ser neutros, profesionales y comprensibles para cualquier desarrollador actual o futuro.

11. **No mezcles idiomas en comentarios:** Define un idioma oficial para el repositorio (usualmente inglés en proyectos internacionales o open source) y sé estricto. La inconsistencia dificulta la lectura y el parsing automático.

Estilo, formato y convenciones

12. **Sigue una guía de estilo consistente:** Alinea con las convenciones del lenguaje y del equipo. Usa mayúsculas/minúsculas, puntuación y estructura uniforme en todos los archivos.
13. **Diferencia y gestiona marcadores temporales:** Usa `TODO`, `FIXME`, `HACK`, `NOTE` solo con contexto claro. Ej: `TODO: refactorizar cuando se migre a v2.0 (ticket-482)`. Establece una política de revisión periódica para evitar que se acumulen.
14. **Prefiere comentarios de bloque para lógica extensa:** Los comentarios en línea (`//`) deben ser breves. Para explicaciones largas, usa bloques (`/* */` o triple comilla) antes del segmento relevante, no intercalados.

Mantenimiento y calidad

15. **Trata los comentarios como código en code reviews:** Revisa precisión, vigencia, claridad y adherencia a convenciones. Un comentario mal escrito o desalineado debe generar comentarios de revisión igual que un bug.
16. **Aprovecha generación automática de documentación:** Usa herramientas compatibles con los comentarios del lenguaje (Doxygen, Sphinx, TypeDoc, Swagger, etc.) para mantener la documentación técnica sincronizada sin esfuerzo manual duplicado.
17. **Elimina comentarios de depuración o experimentales antes de mergear:** `console.log`, `print`, marcadores de prueba o banderas temporales no deben llegar a la rama principal. Usa features flags o logging estructurado en su lugar.

💡 Regla de oro práctica

Si al leer un comentario te preguntas "¿esto aún es cierto?", probablemente ya no lo sea. Actualízalo, refuta el código o elimínalo.

3. Trabaja con commits atómicos:

```
# Después de cada avance funcional  
git add .
```

```
git commit -m "feat: implementa lexer para tokens numéricos"
```

```
git push origin feature/desarrollo-inicial
```

4. **Finaliza con un Pull Request** hacia `main`, revisa los cambios y fusionalos.

5. **Actualiza tu portafolio:** Agrega el enlace al proyecto en la carpeta correspondiente de `tu-usuario.github.io`, haz commit y push. GitHub Pages actualizará la vista automáticamente.

5. Plantilla Obligatoria de README.md (Por Proyecto)

Cada repositorio de proyecto **debe** contener un `README.md` en su raíz con esta estructura exacta:

[Nombre del Proyecto]

Materia: [Nombre] | **Periodo:** [YYYY-N] | **Estado:** Completado/En desarrollo

Equipo de trabajo (De ser necesario)

- [Participante 01](URL al perfil del participante 01)
- [Participante 02](URL al perfil del participante 02)
- [Participante 03](URL al perfil del participante 03)

Capturas / Demo

![Vista principal](docs/screenshots/main.png)

> [Demo en vivo] | [Video/GIF opcional]

Funcionalidad

- [] Funcionalidad 1: Descripción breve [Commit](URL al hito)
- [] Funcionalidad 2: Descripción breve [Commit](URL al hito)
- [] Funcionalidad 3: Descripción breve [Commit](URL al hito)

Tecnologías

`Lenguaje` | `Framework/Librería` | `Base de datos` | `Herramienta CI/CD`

Ejecución

Instrucciones paso a paso

git clone <url-repositorio>

cd <nombre-proyecto>

comando de instalación

comando de ejecución

Métricas de Progreso

Indicador	Valor
Commits totales	[Número]
Issues/PRs fusionados	[X/Y]
Cobertura de pruebas	[%]
Última actualización	[YYYY-MM-DD]

Reflexión y Aprendizajes

- Habilidades desarrollada
- Qué funcionó bien:
- Qué se podría mejorar:
- Conceptos clave aplicados de la materia:

Requisitos:

- Las capturas deben guardarse en `docs/screenshots/`.
- La sección de reflexión es obligatoria y se evalúa con rúbrica.
- No elimines el historial de commits; refleja tu proceso de aprendizaje.

6. Estadísticas y Métricas de Progreso

El portafolio evalúa tu evolución, no solo el producto final. Se recomienda:

- Usar **GitHub Issues** para planificación y **Pull Requests** para integración.
- Actualizar manualmente la tabla de métricas en cada entrega.

- Opcional: Integrar [GitHub Readme Stats] (<https://github.com/anuraghazra/github-readme-stats>) o [Metrics by lowlighter] (<https://github.com/lowlighter/metrics>) para generar badges automáticos de lenguaje, commits y actividad.
- Registrar hitos: `v0.1-alpha`, `v0.5-beta`, `v1.0-release`.

7. Comandos Clave de Git (Referencia Rápida)

Acción	Comando
Clonar repositorio	git clone <url>
Ver estado de cambios	git status
Añadir archivos al stage	git add . o git add <archivo>
Confirmar cambios	git commit -m "mensaje descriptivo"
Subir a GitHub	git push origin <nombre-rama>
Descargar actualizaciones	git pull origin main
Crear rama nueva	git checkout -b <nombre-rama>
Cambiar de rama	git checkout <nombre-rama>
Ver historial compacto	git log --oneline --graph
Etiquetar versión	git tag v1.0.0 → git push --tags

Para una revisión detallada de los comandos de github, puedes revisar la documentación oficial en: <https://docs.github.com/en>

Buenas prácticas:

- Mensajes de commit en presente e imperativo: `feat: agrega validación de entrada`, `fix: corrige loop infinito en parser`.
- Nunca hagas `commit` directo a `main` en proyectos finales o grupales.
- Mantén ramas limpias; elimina ramas fusionadas con `git branch -d <rama>`.

8. Entrega, Revisión y Rúbrica

Entrega

- Publicar el enlace público de tu portafolio (`https://tu-usuario.github.io`) en la plataforma de la materia en la actividad indicada.

- Cada proyecto debe tener su repo público, README completo y estructura de carpetas `Periodo - Materia` vinculada.

Revisión

- Se evaluará automáticamente la existencia de `README.md`, capturas en `docs/screenshots/` y enlaces funcionales.
- Se realizarán revisiones por pares usando un checklist estandarizado.
- La retroalimentación se dejará vía `Pull Requests` o `GitHub Discussions`.

Rúbrica de Evaluación (100 pts)

Criterio	Peso	Indicadores
Documentación técnica	25%	README completo, capturas claras, instrucciones ejecutables
Calidad de código	20%	Estructura, comentarios, manejo de errores, estilo consistente
Métricas y progreso	15%	Tabla actualizada, hitos cumplidos, uso de Issues/PRs
Reflexión pedagógica	20%	Aprendizajes documentados, errores analizados, conexión con teoría
Historial de versionado	10%	Commits atómicos, mensajes descriptivos, ramas funcionales
Presentación/demo	10%	Demo funcional, defensa técnica, respuestas a preguntas

9. Checklist Final de Verificación

- ☐ Repositorio `tu-usuario.github.io` creado y GitHub Pages activo.
- ☐ Estructura de carpetas con nomenclatura `Periodo - Materia` implementada.
- ☐ Cada proyecto tiene repositorio independiente vinculado correctamente.

- [] `README.md` por proyecto incluye: capturas, funcionalidad, ejecución, métricas y reflexión.
- [] Historial de commits limpio, con ramas y mensajes descriptivos.
- [] Tabla de métricas actualizada en cada entrega.
- [] Enlace público del portafolio compartido y accesible.

🔴 **Nota final:** Este portafolio es tuyo. Cuídalo, actualízalo y úsalo como tu carta de presentación técnica. Los estándares aplicados aquí reflejan las expectativas reales de la industria del desarrollo de software.